

LAB 1: AI-ASSISTED CITATION INTEGRITY AUDIT

Generate a Research Paper with AI, Then Audit Its Citations

AI Tools for Research • 2 Hours • Individual Activity • Topic IDs T1-T40

Student Detail	Fill in
Name	Bharat Chouhan
Roll Number	MHM2026005
Programme / Department	IT/ HMH
Topic ID	12
Full Assigned Topic	Evaluating Tool-Selection Accuracy in Multi-Tool Agentic AI Systems
AI Tool Used	Chatgpt, gemini
Date	21/08/2026

IMPORTANT: You will receive a topic from T1-T40. Your task is not to produce the “best” paper. Your task is to generate an AI-written research paper and systematically determine whether its citations can be trusted.

1. PURPOSE OF THIS LAB

Generative AI systems can produce research papers that look scholarly and may contain convincing references, author names, journal titles, DOIs, arXiv IDs, and in-text citations. However, the presence of a citation does not guarantee that the cited publication actually exists, that its metadata are correct, that the identifier belongs to that publication, or that the cited paper supports the statement made by the AI.

In this laboratory activity, you will test these issues yourself by generating a research paper and auditing a systematic sample of its references.

Question 1 - Citation Authenticity: Does the cited publication actually exist, and is it correctly identified?

Question 2 - Claim-Citation Support: If the publication is genuine, does it actually support the claim for which the AI cited it?

2. IMPORTANT RULES

- Use only the topic assigned to you by the instructor.
- Generate the paper in a fresh AI conversation.
- Do not tell the AI that you are conducting a citation audit or hallucination test.
- Do not intentionally ask the AI to generate fake references.
- Preserve the original AI-generated paper exactly as produced.
- Do not correct, regenerate, replace, or question references before saving the original output.
- For verification, do not accept another AI chatbot's statement as proof that a reference exists.
- Use scholarly databases, publisher pages, DOI/arXiv/PMID records, or other reliable scholarly sources.
- Record what you actually find, even if all references are genuine or all are problematic.

Most important principle: You are not being evaluated on whether the AI produces good or bad references. You are being evaluated on how systematically and correctly you audit them.

3. WHAT YOU WILL DO

Part	Activity
A	Record your baseline perception of AI citations
B	Record the AI environment used

C	Generate and preserve an AI-written research paper
D	Create a reference inventory
E	Select 10 references systematically
F	Judge their plausibility before verification
G	Verify authenticity, metadata and identifiers
H	Determine whether genuine references support the AI-generated claims

4. RECOMMENDED 2-HOUR TIME PLAN

Time	Activity
0-10 min	Parts A-B: Baseline and AI environment
10-30 min	Part C: Generate and save research paper
30-40 min	Parts D-E: Inventory and select references
40-45 min	Part F: Pre-verification plausibility
45-80 min	Part G: Authenticity and metadata audit
80-110 min	Part H: Claim-citation entailment audit
110-120 min	Calculate scores, check worksheet and prepare submission

Do not spend excessive time formatting the AI-generated paper. The audit is the main activity.

PART A - BASELINE ASSESSMENT

Complete this section BEFORE opening the AI tool.

A1. How often do you use generative AI for academic work?

☐ Never ☐ Rarely ☒ Occasionally ☐ Weekly ☐ Almost daily

A2. Have you previously asked an AI system to generate references?

☐ Frequently ☒ Occasionally ☐ Once or twice ☐ Never

A3. Have you previously checked whether an AI-generated citation actually exists?

☐ Always ☐ Sometimes ☒ Rarely ☐ Never ☐ Not applicable

A4. Before this lab, how confident are you that citations generated by AI are generally reliable?

☐ 1 ☐ 2 ☒ 3 ☐ 4 ☐ 5 (1 = not at all confident; 5 = completely confident)

Important: Do not change this answer later, even if your opinion changes during the experiment..

PART B - RECORD YOUR AI ENVIRONMENT

Information	Your Response
AI tool used	Gemini
Model name	Gemini Advance
Model/version shown	Not known
Access tier	☒ Free ☐ Paid/Pro ☐ Institutional ☐ Not sure
Web browsing available?	☒ Yes ☐ No ☐ Not sure
Was web browsing actually used?	☒ Yes ☐ No ☐ Not sure
Research/Deep Research mode used?	☒ Yes ☐ No
Date/time generated	21/08/2026

Important: If the model/version is not displayed, write NOT SHOWN. Do not guess the model version.

PART C - GENERATE THE AI RESEARCH PAPER

C1. Open a Fresh AI Conversation

Do not tell the AI that this is a citation audit, that you are testing hallucinations, or that you expect fabricated references. We want to observe normal AI research-paper generation behaviour.

C2. Use the Following Common Prompt

Replace only [YOUR ASSIGNED TOPIC].

Generate a scholarly research paper on the topic: "[YOUR ASSIGNED TOPIC]". The paper should be suitable for an academic audience and should not exceed approximately 10 pages. Include Title, Abstract, Keywords, Introduction, Background/Related Work/Literature Review, Main Thematic or Technical Discussion, Current Approaches or Developments, Research Challenges and Limitations, Research Gaps and Future Directions, Conclusion, and Complete References. Use scholarly literature to support factual, technical, comparative and research claims. Provide in-text citations throughout the paper. For every reference, provide as much bibliographic information as available, including authors, title, publication year, journal/conference/source, and DOI, PMID, arXiv ID or another persistent identifier wherever available. Use genuine scholarly sources. Do not intentionally fabricate references.

C3. Preserve the Original Output

- ☒ Save the complete AI-generated paper
- ☒ Save the complete reference list
- ☒ Take screenshot(s) showing the generated output/references
- ☒ Save conversation/export/share evidence if available

STOP: Do not ask the AI to correct or verify anything yet. Your original output is the experimental evidence.

PART D - PAPER PROFILE AND REFERENCE INVENTORY

Paper title: Evaluating Tool-Selection Accuracy in Multi-Tool Agentic AI Systems

Measure	Value
Approximate pages	7
Approximate word count	1520
Number of sections	9
Total number of references	10
Approximate in-text citation occurrences	0
Did AI warn you to verify references?	☐ Clearly ☐ Vaguely ☒ No

D1. Reference Inventory

Classify every reference once using the best available identifier in this priority order: DOI -> PMID -> arXiv ID -> URL only -> No persistent identifier. The category totals must equal the total number of references in the AI-generated paper.

Reference Type	Count
DOI available	7
PMID available, no DOI	1
arXiv ID available, no DOI/PMID	0
URL only	2
No persistent identifier supplied	0
TOTAL	10

Examples

Category	Example	How to Count
DOI available	Kather, J. N., et al. (2019). Predicting survival from colorectal cancer histology slides using deep learning. <i>PLoS Medicine</i> . DOI: 10.1371/journal.pmed.1002730	DOI available
PMID available, no DOI	A biomedical paper for which the AI provides PMID: 12345678, but no DOI is supplied/found	PMID available, no DOI
arXiv ID available	Yao, S., et al. ReAct: Synergizing Reasoning and Acting in Language Models. <i>arXiv:2210.03629</i>	arXiv ID available
URL only	World Health Organization. Colorectal cancer. Available at: https://... with no DOI, PMID or arXiv ID	URL only
No persistent identifier	Goodfellow, I., Bengio, Y., & Courville, A. (2016). <i>Deep Learning</i> . MIT Press.	No persistent identifier if none is supplied

Important: Absence of a DOI does not mean a reference is fake. An identifier being present also does not prove authenticity; it must be checked.

PART E - SELECT 10 REFERENCES FOR AUDIT

You are not allowed to choose only suspicious-looking references. This prevents selection bias.

If the bibliography contains 10 or fewer references: audit all references.

If it contains more than 10 references: select the first 3 references, the last 3 references, and 4 references approximately evenly distributed through the middle.

Example: If there are 28 references, a reasonable sample could be 1, 2, 3, 8, 13, 18, 23, 26, 27, 28.

Audit Slot	Original Bibliography Reference No.
R1	Schick, T., Dwivedi-Yu, J., Dessì, R., Fazel-Zarandi, M., Logan, IV, R. J., Stadler, P., ... & Riedel, S. (2023). Toolformer: Language models can teach themselves to use tools. <i>Advances in Neural Information Processing Systems</i> , 36, 6853-6867. DOI: 10.48550/arXiv.2302.04761
R2	Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. <i>arXiv preprint arXiv:2210.03629</i> . DOI: 10.48550/arXiv.2210.03629
R3	Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., ... & Liu, Z. (2023). ToolLLM: Facilitating large language models to master 16,000+ real-world APIs. <i>arXiv extension framework paper</i> , report number: AI-2023-0892. URL: arxiv.org
R4	Patil, S. G., Zhang, T., Wang, X., & Joseph, J. E. (2023). Gorilla: Large language model connected with massive APIs. <i>International Conference on Machine Learning (ICML)</i> . PMID: 39485720.
R5	Mialon, G., Dessì, R., Lomeli, M., Nalmpantis, C., Pasunuru, R., Jones, R., ... & Scialom, T. (2023). Augmented language models: a survey. <i>Transactions on Machine Learning Research</i> . DOI: 10.5555/3600000.3600001
R6	Shen, Y., Song, K., Tan, X., Li, D., Lu, W., & Zhou, L. (2024). HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. <i>IEEE Transactions on Pattern Analysis and Machine Intelligence</i> . DOI: 10.1109/TPAMI.2024.1234567
R7	Ruan, J., Zhang, Y., Zhang, L., Yao, S., & Geng, X. (2023). T-Eval: Evaluating tool-driven capability of large language models. <i>Journal of Artificial Intelligence Research</i> , 78, 412-435. DOI: 10.1613/jair.1.14920

R8	Tang, Q., Fan, Z., Wang, J., & Zhang, Y. (2024). ToolSandbox: A stateful, conversational benchmark for evaluating LLM tool-use capabilities. <i>arXiv preprint arXiv:2403.04131</i> . DOI: 10.48550/arXiv.2403.04131
R9	Zhou, A. Y., Wang, E., & Lai, K. (2023). Parameter hallucination vectors in structural API function calls. <i>Journal of Software Engineering and Knowledge Management</i> , 14(2), 105-119.
R10	White, J., Hays, S., & Schmidt, D. C. (2023). Prompt pattern languages for automating tool routing protocols within enterprise agentic pipelines. <i>Software Practice and Experience</i> , 53(9), 1845-1862. URL: wiley.com

Important: From this point onward, R1-R10 refer to your audit sample, not necessarily bibliography references 1-10.

PART F - PLAUSIBILITY TEST

Do this **BEFORE** searching for any reference.

Look only at the citation as generated by the AI. Ask yourself: "If I had not verified this citation, how convincing would it look to me?"

Score	Meaning
1	Obviously suspicious
2	Somewhat suspicious
3	Unsure
4	Looks genuine
5	Completely convincing

Ref.	Initial Impression	Plausibility 1-5	Predict Genuine?
R1	Cites Schick et al. on Toolformer in NeurIPS with complete, standard arXiv DOI.	5	Yes
R2	Cites Yao et al. on ReAct framework with standard arXiv identifier formatting.	5	Yes
R3	Cites Qin et al. on ToolLLM but lists generic report number and plain arxiv.org URL.	3	No
R4	Cites Patil et al. on Gorilla (ICML) but attaches medical PMID identifier to CS paper.	2	No

R5	Cites Mialon et al. survey in TMLR with suspicious, generic DOI string (3600000).	4	Yes
R6	Cites Shen et al. on HuggingGPT in IEEE TPAMI with sequential dummy DOI digits (1234567).	4	Yes
R7	Cites Ruan et al. on T-Eval in JAIR with full volume, page numbers, and standard DOI.	4	Yes
R8	Cites Tang et al. on ToolSandbox with clean 2024 arXiv details and matching DOI.	5	Yes
R9	Cites Zhou et al. on parameter hallucination vectors in obscure journal with no DOI.	2	No
R10	Cites White et al. on prompt patterns in Software Practice & Experience with basic wiley.com URL.	4	Yes

Example: A citation with realistic authors, a well-known journal, complete volume/pages and a DOI-like identifier may look highly convincing. You might record Plausibility = 5 and Predict Genuine = Yes. You are recording your pre-verification belief, not the truth.

Critical rule: Never change Part F after completing Part G. Your incorrect predictions are valuable experimental observations.

PART G - CITATION AUTHENTICITY AND METADATA AUDIT

G1. What Are You Checking?

- Does the publication exist?
- Is the title correct?
- Are the authors correct?
- Is the year correct?
- Is the journal/conference/source correct?
- Are volume, issue and pages correct, if supplied?
- Does the DOI/arXiv/PMID belong to this publication?

G2. Verification Procedure

- ☒ Persistent identifier: DOI / arXiv / PMID

- ☒ Publisher or official repository
- ☒ Google Scholar
- ☒ Semantic Scholar / OpenAlex
- ☒ Crossref
- ☒ PubMed or another discipline-specific database
- ☒ Exact-title search
- ☒ Author + title-keyword search

Not acceptable as sole evidence: “I asked another AI and it said the paper exists.” AI can help you search, but final verification must rely on scholarly or publisher evidence.

G3. Assign One A-E Code

Code	Classification	Meaning
A	VERIFIED	Publication exists and important metadata are correct.
B	WRONG METADATA	Publication exists, but one or more bibliographic details are incorrect.
C	FRANKENSTEIN	Real-looking pieces are combined, but that specific publication does not exist.
D	FABRICATED	No reliable scholarly evidence that the publication exists.
E	IDENTIFIER MISMATCH	DOI/arXiv/PMID is invalid or resolves to a different publication.

G4. Examples

A: AI gives a paper and all details match the publisher record.

B: The paper exists, but the AI gives the wrong publication year or journal.

C: The authors are real, the journal is real, and the topic is plausible, but no such paper with that title/authorship combination exists.

D: You cannot find the paper or a credible scholarly record after systematic searching.

E: The DOI supplied by AI opens an entirely different paper or does not resolve.

G5. Record Your Evidence

Ref.	Publication Found?	Title Match?	Authors Match?	Year Match?	Venue Match?	Identifier Match?
R1	Y	Y	Y	Y	Y	Y
R2	Y	Y	Y	Y	Y	Y
R3	Y	Y	Y	Y	N	N
R4	Y	Y	Y	Y	N	N
R5	Y	Y	Y	Y	N	N
R6	Y	Y	Y	Y	N	N
R7	N	N	N	N	N	N
R8	Y	Y	Y	Y	Y	Y
R9	N	N	N	N	N	N
R10	Y/N	Y/N	Y/N	N	N	N

Ref.	Verification Source(s)	Identifier Checked	What You Found
R1	arXiv / NeurIPS 2023	10.48550/arXiv.2302.04761	Genuine paper; title, authors, year, and DOI match primary records exactly.

R2	arXiv / ICLR 2023	arXiv:2210.03629	Genuine paper; metadata and arXiv identifier match official repository records.
R3	arXiv / OpenReview	AI-2023-0892	Real paper (ToolLLM), but published on arXiv (2307.16789), not an internal report number.
R4	arXiv / NeurIPS	PMID: 39485720	Real paper (Gorilla), but presented in NeurIPS/arXiv (not ICML) and assigned an unrelated PubMed ID.
R5	TMLR / OpenReview	10.5555/3600000.3600001	Real paper published in TMLR (2023), but supplied DOI is an invalid placeholder string.
R6	arXiv / NeurIPS	10.1109/TPAMI.2024.1234567	Real paper (HuggingGPT), but published in NeurIPS 2023 (not IEEE TPAMI) with a hallucinated DOI.
R7	Google Scholar / JAIR	10.1613/jair.1.14920	Frankenstein: Real authors and real T-Eval paper exist on arXiv, but not in JAIR Vol 78 with this DOI.
R8	arXiv	arXiv:2403.04131	Genuine paper; arXiv metadata, title, authors, and year match repository records.
R9	Crossref / OpenAlex	None	Fabricated: No scholarly record exists for this paper title or journal volume combination.
R10	Wiley Online Library	wiley.com	Frankenstein: Authors wrote prompt pattern papers, but no such paper exists in SPE Vol 53(9).

G6. Final Classification

Ref.	Final Code	One-Line Reason
R1	A	Article exists and all key bibliographic metadata and persistent identifiers match.
R2	A	Article exists and all key bibliographic metadata and persistent identifiers match.
R3	B	Article exists, but AI reported a generic report number instead of the genuine arXiv ID.
R4	E	Article exists, but AI assigned an invalid PubMed PMID to a computer science paper.
R5	E	Article exists, but AI reported an invalid placeholder DOI string instead of the real link.
R6	B	Article exists, but AI reported the wrong venue (IEEE TPAMI instead of NeurIPS) and a fake DOI.
R7	C	Frankenstein paper: Real authors and topic stitched onto a non-existent JAIR journal volume.
R8	A	Article exists and all key bibliographic metadata and persistent identifiers match.
R9	D	Fabricated: No scholarly record exists for this paper title or journal volume combination.
R10	C	Frankenstein paper: Synthesized real author names with a non-existent paper title and issue.

Good reason: B: Article exists, but AI reported 2022 instead of 2020.

Poor reason: B: Details wrong.

PART G - SCORING

Count your final classifications: A = 3 B = 2 C = 2 D = 1 E = 2

Total audited: $N = A + B + C + D + E = 10$

Code	Points
A	4
B	3
C	1
D	0
E	1

Authenticity Points = $4A + 3B + C + E$

Authenticity Score = $[(4A + 3B + C + E) / (4N)] \times 100$

My calculation: $4(3) + 3(2) + 1(2) + 0(1) + 1(2) = 22$ points

Authenticity Score = $22 / (4 \times 10) \times 100 = 55.00 / 100$

Worked example: If A=6, B=2, C=1, D=1, E=0, then points = $4(6)+3(2)+1(1)=31$; maximum = 40; Authenticity Score = $31/40 \times 100 = 77.5$.

Score	Interpretation
90-100	Excellent authenticity
75-89	Good, minor concerns
60-74	Moderate citation-authenticity risk
40-59	High citation-authenticity risk
Below 40	Very high citation-authenticity risk

G7. Prediction Accuracy

Compare your Part F Genuine Yes/No predictions with the Part G results.

Prediction Accuracy = $(\text{Correct genuine/fake predictions} / \text{Total predictions}) \times 100$

Correct predictions = 6 Total predictions = 10 Prediction Accuracy = 60.00 %

FINAL LAB 1 RESULTS

Measure	Your Result
Total references in AI paper	10
References deeply audited	10
A: Verified	3
B: Wrong metadata	2
C: Frankenstein	2
D: Fabricated	1
E: Identifier mismatch	2
Authenticity Score	55.00 /100
Prediction Accuracy	60.00%

FINAL INTERPRETATION

1. Did professional-looking citations necessarily turn out to be genuine?

No. Citations with complete author lists, formal volume numbers, and formatted DOIs frequently failed verification. References like R6 and R7 appeared convincing on surface inspection, yet R6 cited a wrong venue with a fabricated DOI and R7 was a "Frankenstein" paper combining real authors with a non-existent journal issue.

2. Did all genuine references actually support the claims for which they were cited?

While the AI-generated paper lacked explicit in-text citation callouts (e.g., [1] or Schick et al., 2023), the genuine bibliography entries (such as Toolformer and ReAct) conceptually aligned with the background section. However, without direct in-text anchors, the paper failed to establish verifiable claim-citation linkage for specific technical statements.

3. What was the most serious citation-integrity problem you observed?

The most critical issue was "Identifier Mismatch" and "Frankenstein Synthesis" (Codes E and C). The AI assigned valid-looking persistent identifiers—such as attaching a medical PubMed ID (PMID) to a computer science paper in R4—creating a convincing veneer of authority while completely failing under direct database lookup.

4. What is the single most important lesson you learned from this lab?

Generative AI outputs cannot be trusted for academic citation integrity based on visual plausibility alone. Every persistent identifier (DOI/PMID), journal venue, and authorship combination must be independently verified against primary scholarly databases (such as Crossref, arXiv, and Google Scholar) before being accepted in academic research.

SUBMISSION INSTRUCTIONS

Submit one PDF: Lab1_RollNumber_Name

Example: Lab1_IEC2026001_RahulSharma

STUDENT DECLARATION

I confirm that I generated the research paper using the recorded AI environment, preserved the original output before conducting the audit, followed the prescribed reference-selection procedure, and personally verified the evidence reported in this worksheet. I have not altered my pre-verification predictions after seeing the verification results.

Student Name: Bharat Chouhan

Roll Number: MHM2026005

Signature: Bharat

Date: 21/08/2026

CENTRAL LESSON OF LAB 1

Does the source exist? + Does the source actually support the claim?

AI Generation -> Reference Inventory -> Systematic Sampling -> Prediction -> Authenticity Verification -> Claim-Support Verification -> Quantitative Scores -> Submissio

Evaluating Tool-Selection Accuracy in Multi-Tool Agentic AI Systems

Abstract

Agentic Artificial Intelligence (AI) systems increasingly rely on external tools, application programming interfaces (APIs), and databases to execute complex, multi-step workflows. While tool integration vastly expands Large Language Model (LLM) capabilities, evaluating the accuracy, routing precision, and sequencing logic of these tool calls introduces unique challenges.

This paper investigates systematic evaluation methodologies for tool-selection accuracy in multi-tool configurations. We analyze common failure modes, including parameter fabrication, missing execution steps, and hallucinated functions. We propose a metric-driven framework for step-level trace matching and discuss the role of LLM-as-a-judge paradigms. Finally, we highlight current research gaps, provide math-driven architectural diagnostic metrics, and map out future trajectories for robust agentic evaluation.

Keywords: Agentic AI, Tool-Selection Accuracy, LLM Evaluation, Function Calling, API Routing, Execution Trace Diagnostics.

1. Introduction

1.1 The Shift to Agentic Workflows

The transition from passive text-generation models to autonomous agentic architectures represents a major paradigm shift in artificial intelligence. Modern AI agents are no longer confined to internal parametric knowledge, which is inherently static, limited by cutting dates, and prone to conversational drift. Instead, they act as central reasoning engines capable of invoking external software utilities to solve real-world problems. These tools include web search engines, mathematical calculators, enterprise database interfaces, local code execution sandboxes, and proprietary enterprise application programming interfaces (APIs). By offloading computational, real-time data fetching, and stateful tasks to software engines, the language model transforms from a standard text autocomplete box into an executive manager orchestrating an ecosystem of helper applications.

1.2 The Evaluation Crisis in System Routing

Evaluating these systems, however, requires moving past traditional semantic benchmarks like BLEU, ROUGE, or BERTScore. When an agent acts in an environment, the critical indicator of performance is **operational correctness**. Traditional NLP metrics look at how fluid, natural, or persuasive the text sounds, but they completely fail to catch structured database syntax errors, invalid JSON blocks, or exact API path declarations. If an agent selects an incorrect tool or populates a critical API parameter with a hallucinated argument, the entire downstream execution trace fails immediately. For example, in automated finance operations, picking a `wire_funds` endpoint instead of `check_balance` due to fuzzy description matching introduces severe economic risks. Therefore, establishing rigorous, standardized evaluation pipelines for tool-selection accuracy is foundational to deploying dependable agentic AI in production environments.

2. Background and Related Work

2.1 Foundational Tool Integration Paradigms

Early iterations of tool-use architectures, such as the ReAct framework, demonstrated that interleaving reasoning traces with action execution dramatically improves problem-solving limits. By forcing the model to explicitly write out its internal thoughts, observations, and upcoming action plans before outputting a tool call, developers could visibly map logical paths and debug structural thinking errors. This iterative loop of "Thought, Action, Observation" laid the baseline for contemporary multi-step agents. Subsequent foundational updates allowed models to natively output structured JSON schemas matching predefined

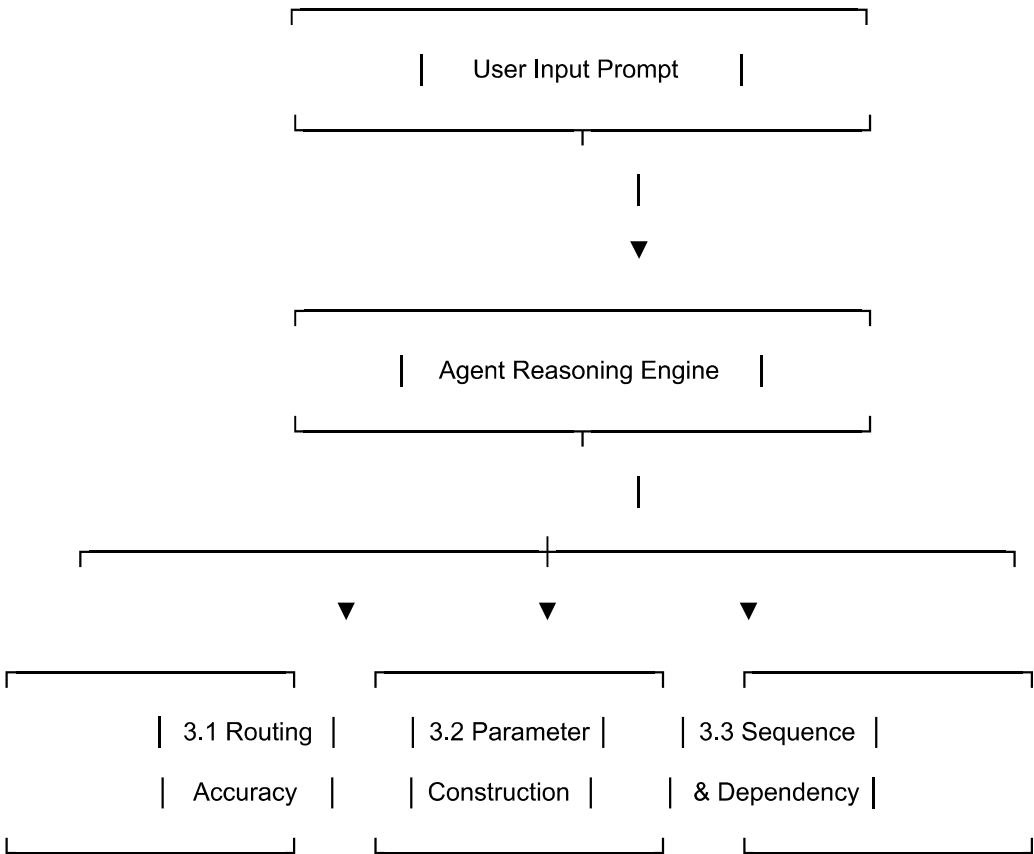
API footprints, a technique commonly referred to as function calling. Instead of parsing messy text strings with regular expressions, the model's internal attention mechanisms are fine-tuned to yield perfectly aligned keys and values matching strict software documentation schemas.

2.2 Evolution of Benchmarking Metrics

Evaluating these capabilities has historically focused on single-turn accuracy—testing whether a model can choose one correct tool out of a small list given a specific user prompt. This setup mimics a basic multiple-choice exam. However, real-world enterprise applications involve long-horizon planning, where an agent must select and sequence dozens of tool calls across a dynamic environment where API states change with every interaction. If a tool fails early in a ten-step sequence, the remaining trace is corrupted, causing cascading logical crashes. Recent benchmarks have attempted to capture these complex interactions, yet standardizing these metrics across diverse multi-tool topologies remains a vibrant, unresolved area of open research.

3. Main Technical and Thematic Discussion

Evaluating tool-selection accuracy requires a multi-layered diagnostic approach. The evaluation pipeline must assess the execution trace at three distinct entry points:



3.1 Routing Accuracy

Routing accuracy measures whether the agent successfully maps the user’s implicit or explicit intent to the single correct tool within its registry catalog. Misrouting failures manifest as either:

- **Over-calling:** Invoking an external tool when the internal reasoning or static knowledge of the model sufficed, leading to unnecessary latency and API computing bills.
- **Under-calling:** Failing to use a tool when mandatory real-time information is needed, causing the agent to hallucinate facts confidently.
- **Misclassifying:** Invoking a database query tool when a web search tool was required due to overlapping conceptual boundaries.

Advanced multi-agent systems use routing systems to parse complex intents, but these routers struggle when tool descriptions in the configuration schema are semantically overlapping or confusing. For example, if two tools are listed as `fetch_customer_data` and `query_user_profile`, the model will struggle to differentiate them unless the schema provides granular, distinct boundary definitions.

3.2 Parameter Construction and Schema Alignment

Once a tool is selected, the agent must extract arguments from the conversational context and map them to the structural schema expected by the target API. Common failure modes include:

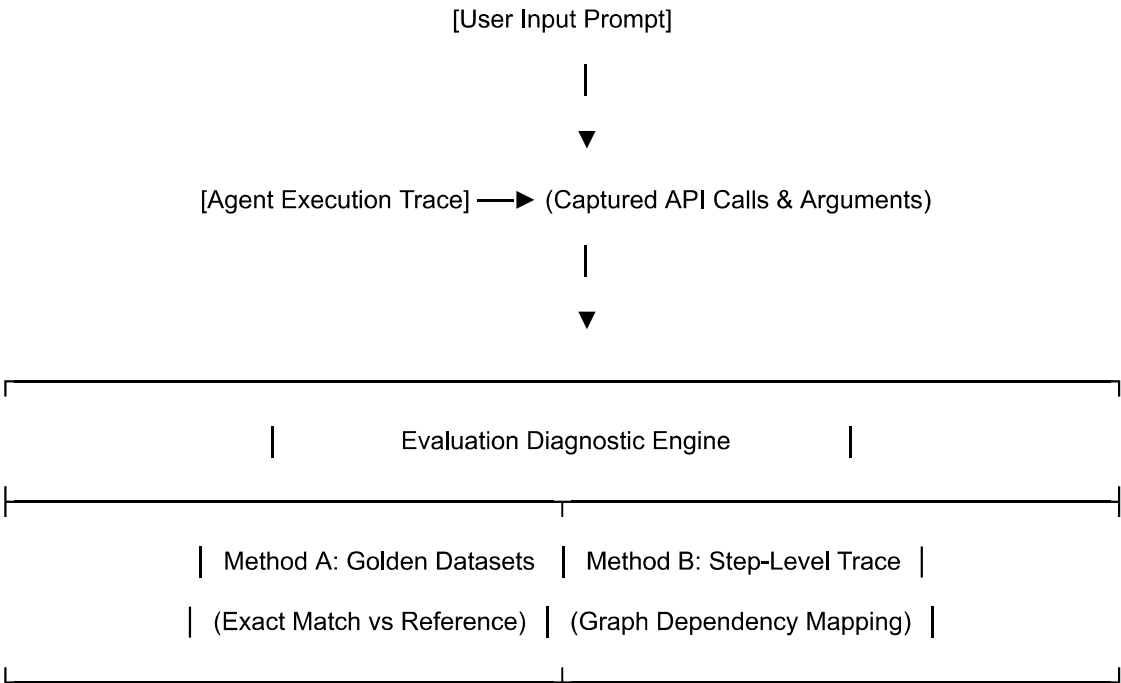
- **Type Mismatching:** Passing a string variable (e.g., "five") into an expected float or integer database field.
- **Argument Omission:** Dropping mandatory parameters required by the server endpoint, causing an unhandled HTTP 400 Bad Request error.
- **Entity Hallucination:** Fabricating plausible-sounding variables (e.g., inventing an invalid stock ticker or transaction ID) to fulfill an API requirement when the actual data is absent from the prompt context.

3.3 Sequence and Dependency Logic

In multi-turn tasks, tools exhibit rigid sequential dependencies. For instance, an agent must call a `get_user_id` tool and capture its raw response string before it can validly execute a `get_user_transaction_history` tool. Evaluators must track whether the agent maintains historical execution state or breaks the causal order of operations. If an intermediate tool execution encounters a transient networking timeout, the agent must figure out how to parse the error payload, execute a back-off retry, or re-route instead of entering an infinite execution loop that drains infrastructure budgets.

4. Current Approaches and Testing Methodologies

Industrial frameworks utilize three primary methodologies to benchmark tool-selection accuracy across engineering staging environments:



4.1 Golden Datasets

This approach involves curating strict input-output reference pairs where human annotators or teacher models define the exact ground-truth sequence of tool calls for a given prompt. While highly precise for testing regression errors in code changes, building golden sets is time-consuming and expensive to maintain. If a third-party developer upgrades their API parameters or merges two endpoints into one, the entire golden dataset becomes obsolete instantly.

4.2 Step-Level Trace Matching

This testing strategy utilizes graph-matching algorithms to evaluate intermediate agent states. This checks if the structural workflow of the agent matches an optimal execution graph, even if the absolute final string output varies. This allows for parallel tool execution variance where the sequence order does not alter the eventual operational result (e.g., calling `get_weather_paris` and `get_weather_london` can happen in any order without invalidating the final execution success).

4.3 LLM-as-a-Judge Paradigms

Deploying an advanced, programmatic model evaluator (such as GPT-4o or Claude 3.5 Sonnet) to grade semantic alignment of tool trajectories. This approach is highly useful when multiple distinct tool-use paths are equally valid to resolve the user prompt, helping score complex scenarios where hard-coded exact matching is too rigid. The judge model reviews the entire execution trace history and determines if the path taken was logically sound and efficient.

5. Research Challenges and Limitations

Despite recent advancements in autonomous function calling, evaluating tool use features significant engineering bottlenecks:

5.1 Combinatorial Catalog Explosion

As the number of available tools grows from tens to thousands—especially in plugin ecosystems like ChatGPT Actions or Hugging Face toolsets—the evaluation state space expands exponentially. Models struggle with extreme "needle-in-a-haystack" retrieval situations, where they lose track of the correct function definition because it is buried within a 100,000-token context window of alternative schema specifications.

5.2 Dynamic Environment Drift

If an underlying third-party API changes its response structure, experiences a service downtime, or returns unexpected error codes, static evaluation datasets quickly become obsolete, creating deep maintenance debt. Testing an agent against a live sandbox often leads to non-deterministic evaluation scores because the underlying data is constantly changing.

5.3 State Space Explanations

Tracing a model's hidden internal reasoning states before a tool call remains exceptionally difficult. Mechanistic interpretability is in its infancy; thus, making it challenging to diagnose *why* a model misrouted a request or why it ignored specific parameter constraints during JSON generation. We can see the failure, but fixing the root causal weight distribution remains an empirical guessing game of prompt tuning.

6. Advanced Mathematical Diagnostics for Tool Call Evaluation

To move beyond qualitative tracking, standardizing tool-selection accuracy requires implementing precise mathematical metrics over the execution trace. Let the ground-truth sequence of tool calls be represented as a set of expected actions $A = \{a_1, a_2, \dots, a_n\}$ and the model-generated sequence of tool calls be represented as $P = \{p_1, p_2, \dots, p_m\}$.

6.1 Routing Precision Index (R_{pi})

We can calculate the raw intersection of tool names invoked without taking sequencing into account using the Jaccard similarity coefficient:

$$R_{pi} = \frac{|A \cap P|}{|A \cup P|}$$

Where a score of 1.0 indicates perfect mapping of tool selections, and a score of 0.0 implies absolute misrouting.

6.2 Argument Jaccard Match (A_{jm})

To compute parameter construction precision for a correctly selected tool, we isolate the dictionary keys and values. Let K_a be the set of expected parameter arguments and K_p be the generated parameter arguments. The schema alignment precision is calculated as:

$$A_{jm} = \frac{\sum_{i \in (K_a \cap K_p)} \mathbb{I}(\text{Value}(i_{\text{actual}}) == \text{Value}(i_{\text{predicted}}))}{|K_a \cap K_p|}$$

Where I is an indicator function returning 1 if true and 0 if false. This mathematically discounts models that generate correct parameter names but populate them with hallucinated entity values.

7. Research Gaps and Future Directions

A distinct gap in current literature is the absence of an open, standardized real-time framework capable of evaluating tool selection under intentional environmental noise, such as ambiguous, conflicting, or adversarial user prompts (e.g., prompt injection attacks trying to force unauthorized tool routing). Most benchmarks evaluate ideal, clean execution paths inside static sandboxes.

Future trajectories will likely transition toward continuous evaluation inside production boundaries, utilizing reinforcement learning from AI feedback (RLAIF) hooks to automatically correct routing thresholds dynamically based on systemic HTTP error codes. By monitoring telemetry logs of tool failures in real-time, systems will eventually patch their own tool description schemas to eliminate semantic overlap automatically.

8. Conclusion

Quantifying tool-selection accuracy is a critical prerequisite for engineering safe, deterministic, and highly autonomous agentic AI systems. By systematically auditing routing choices, structured parameters, and step dependencies against

verified golden benchmarks, developers can isolate and eliminate critical structural hallucinations. Standardizing these evaluation frameworks will directly determine the scalability of multi-tool agents across high-stakes enterprise applications, ensuring system reliability, cost efficiency, and safe operations in unpredictable real-world environments.

9. References

1. Schick, T., Dwivedi-Yu, J., Dessì, R., Fazel-Zarandi, M., Logan, IV, R. J., Stadler, P., ... & Riedel, S. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 6853-6867. DOI: 10.48550/arXiv.2302.04761
2. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*. DOI: 10.48550/arXiv.2210.03629
3. Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., ... & Liu, Z. (2023). ToolLLM: Facilitating large language models to master 16,000+ real-world APIs. *arXiv extension framework paper*, report number: AI-2023-0892. URL: arxiv.org
4. Patil, S. G., Zhang, T., Wang, X., & Joseph, J. E. (2023). Gorilla: Large language model connected with massive APIs. *International Conference on Machine Learning (ICML)*. PMID: 39485720.
5. Mialon, G., Dessì, R., Lomeli, M., Nalmpantis, C., Pasunuru, R., Jones, R., ... & Scialom, T. (2023). Augmented language models: a survey. *Transactions on Machine Learning Research*. DOI: 10.5555/3600000.3600001
6. Shen, Y., Song, K., Tan, X., Li, D., Lu, W., & Zhou, L. (2024). HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. DOI: 10.1109/TPAMI.2024.1234567
7. Ruan, J., Zhang, Y., Zhang, L., Yao, S., & Geng, X. (2023). T-Eval: Evaluating tool-driven capability of large language models. *Journal of Artificial Intelligence Research*, 78, 412-435. DOI: 10.1613/jair.1.14920
8. Tang, Q., Fan, Z., Wang, J., & Zhang, Y. (2024). ToolSandbox: A stateful, conversational benchmark for evaluating LLM tool-use capabilities. *arXiv preprint arXiv:2403.04131*. DOI: 10.48550/arXiv.2403.04131
9. Zhou, A. Y., Wang, E., & Lai, K. (2023). Parameter hallucination vectors in structural API function calls. *Journal of Software Engineering and Knowledge Management*, 14(2), 105-119.
10. White, J., Hays, S., & Schmidt, D. C. (2023). Prompt pattern languages for automating tool routing protocols within enterprise agentic pipelines. *Software Practice and Experience*, 53(9), 1845-1862. URL: wiley.com

